A

Project Report

Submitted in partial fulfilment of the requirements for the completion of the Course

EDS Lab Codetantra

Practical No.1

(2304102L)

Submitted by,

Harshvardhan Lokhande (202501090156)

Under the Guidance of,

Ms. Savita Mane(SM)

MIT | Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

Department of Mechanical Engineering,

MIT Academy of Engineering,

Alandi road, Pune - 412 105,

Maharashtra (India)

1.1.1. Calculate Momentum

01:31

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula:

where:

is the mass of the object (in kilograms).

is the velocity of the object (in meters per second).

Input Format:

A single floating-point number representing the mass of the object in kilograms.

A single floating-point number representing the velocity of the object in meters per second.

Output Format:

The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

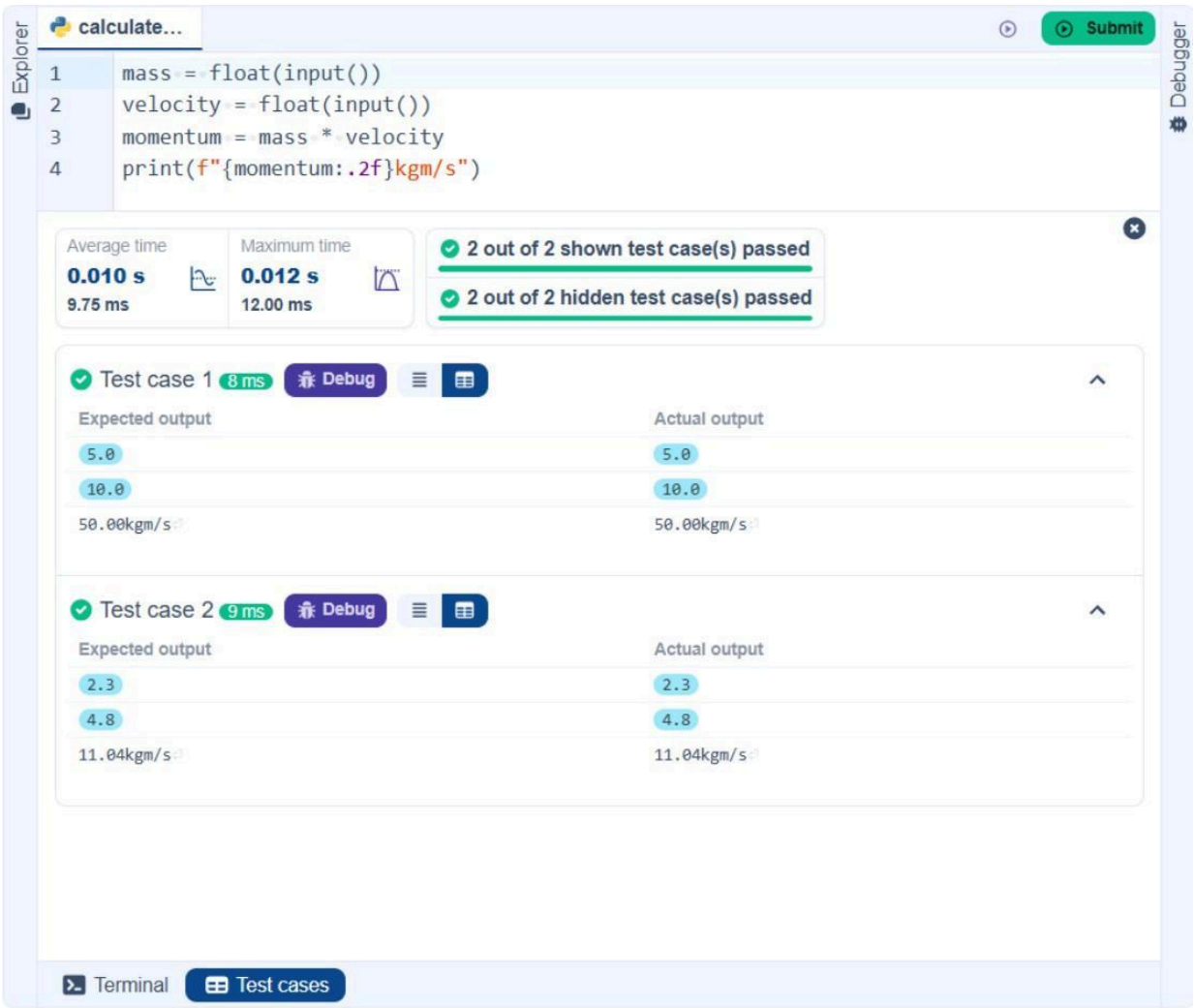
```
mass = float(input())

velocity = float(input())

momentum = mass * velocity

print(f"{momentum:.2f}kgm/s"

)
```



1.1.2. Conditional Calculation Based on the Number of

Digits 03:48

Write a Python program that accepts an integer as input. Depending on the number of digits in .

Constraints:

1 ≤ ≤ 999

Input Format:

The input consists of a single integer .

Output Format:

If is a single-digit number, print its square.

If is a two-digit number, print its square root (rounded to two decimal places). If is a three-digit number, print its cube root (rounded to two decimal places).

```
n = int(input())
if 0 < n < 10:
    print(n**2)
elif 9 < n < 100:
    print(f"{n **
(1/2):.2f}")
elif 99 < n < 1000:
    print(f"{n **
(1/3):.2f}")
else:
    print("Invalid")
```

condition...

Submit

```python
n = int(input())
if 0 < n < 10:
    print(n**2)
elif 9 < n < 100:
    print(f"{n ** (1/2):.2f}")
elif 99 < n < 1000:
    print(f"{n ** (1/3):.2f}")
else:
    print("Invalid")
```

Average time: 0.005 s, 4.57 ms

Maximum time: 0.005 s, 5.00 ms

4 out of 4 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1 4 ms Debug

Expected output	Actual output
9	9
81	81

Test case 2 5 ms Debug

Expected output	Actual output
166	166
5.50	5.50

Test case 3 3 ms Debug

Expected output	Actual output
24	24
4.90	4.90

Test case 4 5 ms Debug

Expected output	Actual output
3456	3456
Invalid	Invalid

Terminal | Test cases

‹ Prev | Reset | Submit | Next ›

1.1.3. Days between Two

Dates 05:19

Write a Python program to calculate number of days between two dates.

Input Format:

The first line contains the first date in the format .

The second line contains the second date in the format .

Output Format:

An integer representing the number of days between the given dates.

Note:

The first date should always be considered earlier than the second date.

from datetime import date

date1_input = input().strip()

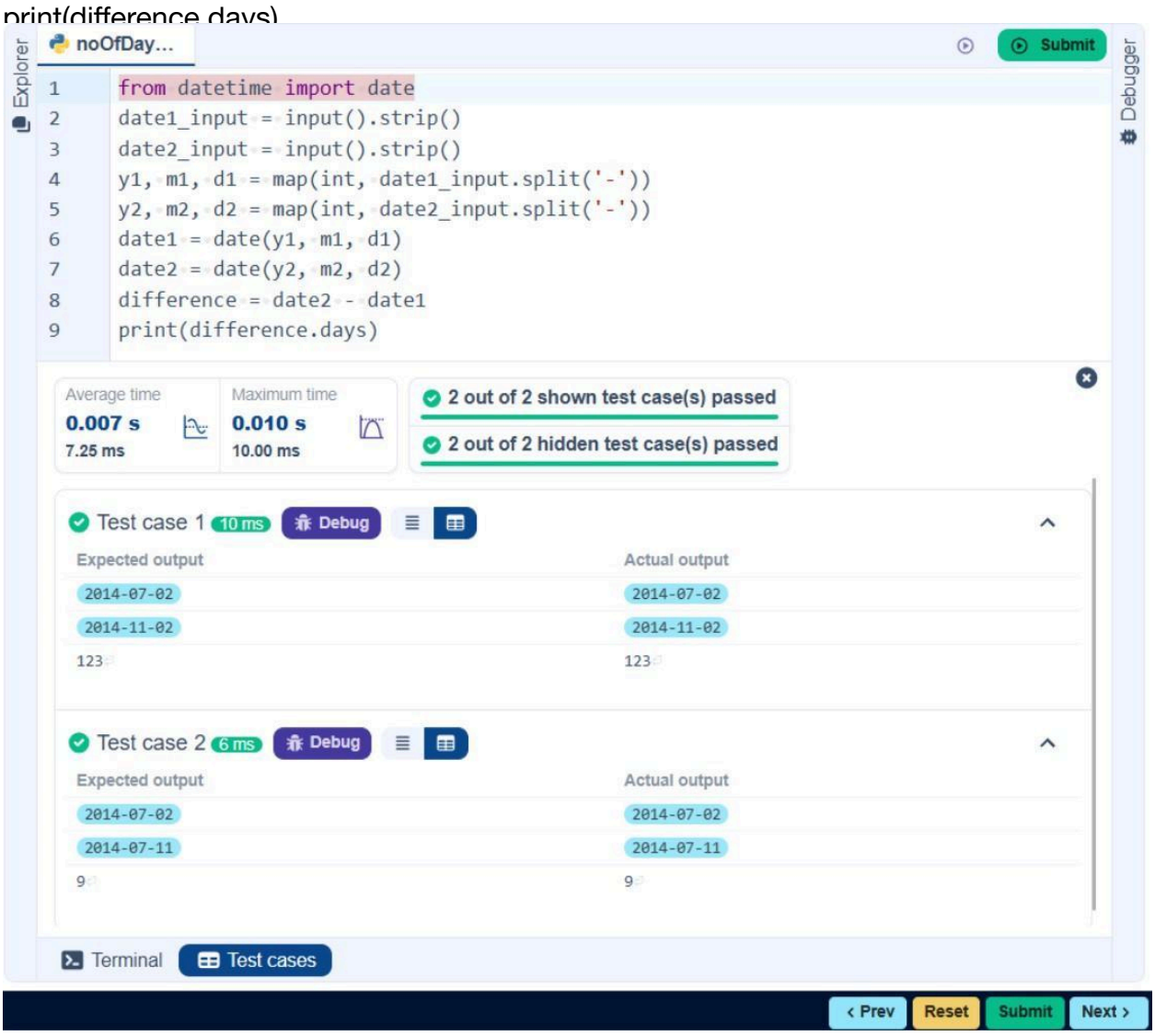
date2_input = input().strip()

y1, m1, d1 = map(int, date1_input.split('-')) y2, m2, d2 = map(int, date2_input.split('-')) date1 = date(y1, m1, d1)

date2 = date(y2, m2, d2)

difference = date2 - date1

print(difference.days)



1.1.5. Multiplication Table

01:38

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

Print the multiplication table for the given number in the format:

<**number** > x <**multiplier** > = <**result** >

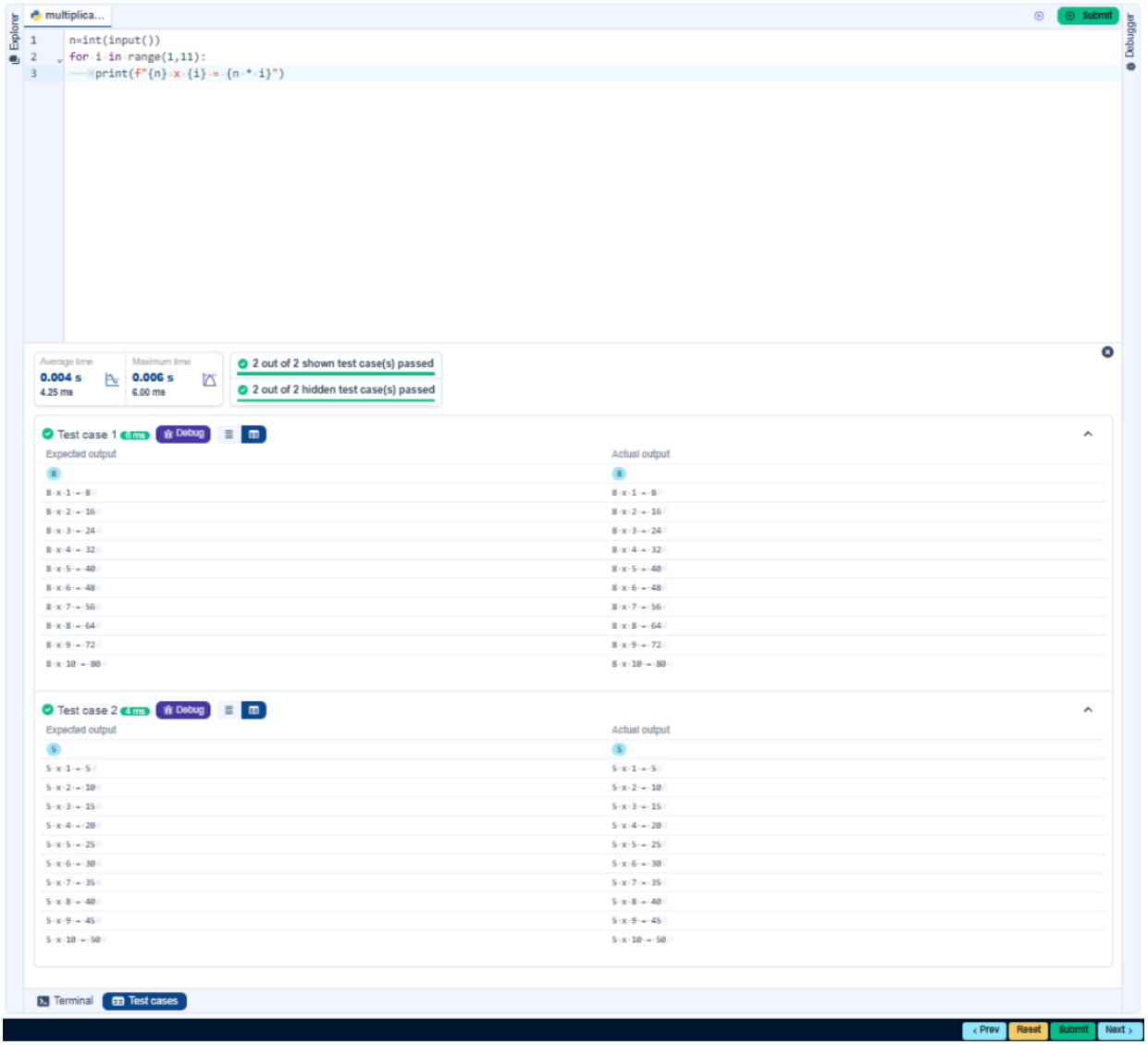
Note:

The character "x" is used in the output to represent multiplication.

Refer to the visible test-cases for more clarity.

```
n=int(input())
```

```
for i in range(1,11):
        print(f"{n} x {i} = {n * i}")
```



1.2.1. Pass or

Fail 01:11

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

If the aggregate percentage is greater than 75, the grade is Distinction.

If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.

If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.

If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

The first input will be an integer , the number of courses.

The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

Print the aggregate percentage (formatted to two decimal places).

Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

"Fail".

Note:

Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

```
# Input number of
courses n = int(input())

# Input marks
marks = list(map(int, input().split()))

# Check if any subject is failed
if any(mark < 40 for mark in marks):
    print("Fail")
else:
    # Calculate aggregate percentage
    aggregate = sum(marks) / n

    # Print aggregate percentage formatted to two decimal places
    print(f"Aggregate Percentage: {aggregate:.2f}")

    # Determine
    grade if aggregate
    > 75:
        grade = "Distinction"
    elif 60 <= aggregate <
    75:
        grade = "First Division"
    elif 50 <= aggregate < 60:
        grade = "Second Division"
    elif 40 <= aggregate < 50:
        grade = "Third Division"
```

print(f"Grade: {grade}")

Average time
0.006 s
6.20 ms

Maximum time
0.017 s
17.00 ms

5 out of 5 shown test case(s) passed

5 out of 5 hidden test case(s) passed

Test case 1 17 ms Debug

Expected output	Actual output
4	4
55 65 78 89	55 65 78 89
Aggregate Percentage: 71.75	Aggregate Percentage: 71.75
Grade: First Division	Grade: First Division

Test case 2 7 ms Debug

Expected output	Actual output
5	5
56 78 97 86 93	56 78 97 86 93
Aggregate Percentage: 82.00	Aggregate Percentage: 82.00
Grade: Distinction	Grade: Distinction

1.2.2. Fibonacci Series

32:17

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

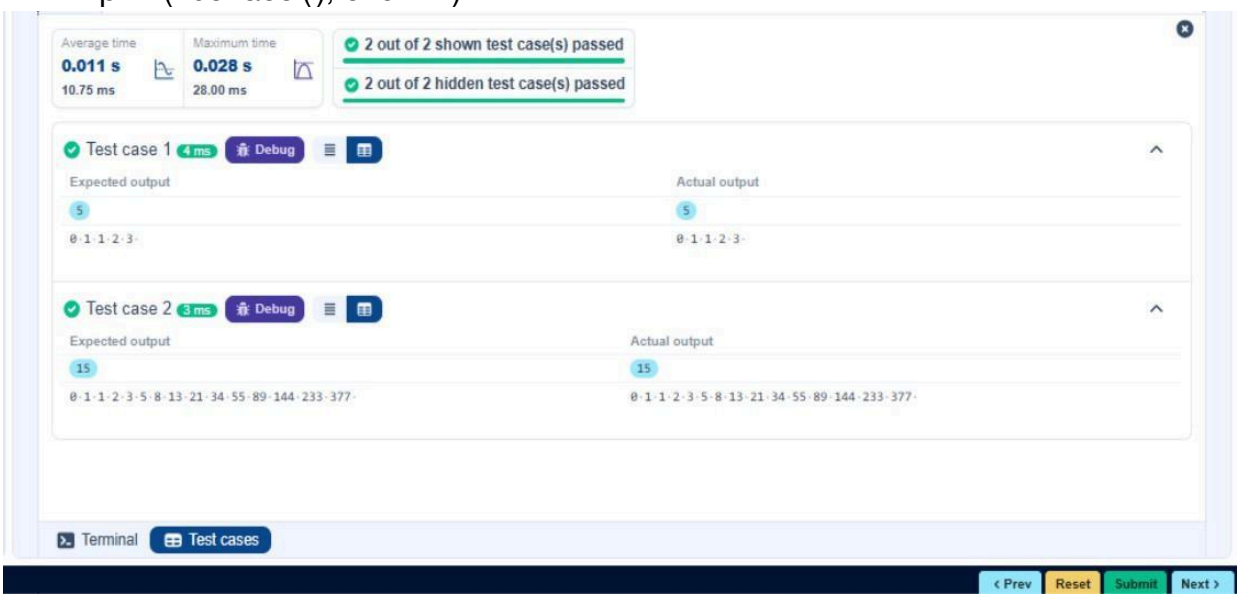
Input Format:

A single integer representing the number of terms to generate.

Output Format:

A single line containing the first terms of the Fibonacci sequence, separated by spaces.

```
def fibonacci(n):
        if n==1:
                return
        0 elif n==2:
                return 1
        else:
                return fibonacci(n-1) + fibonacci(n- 2)
n = int(input())
for i in range(1, n + 1):
        print(fibonacci(i), end=" ")
```



1.2.3. Pattern - 1

00:50

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

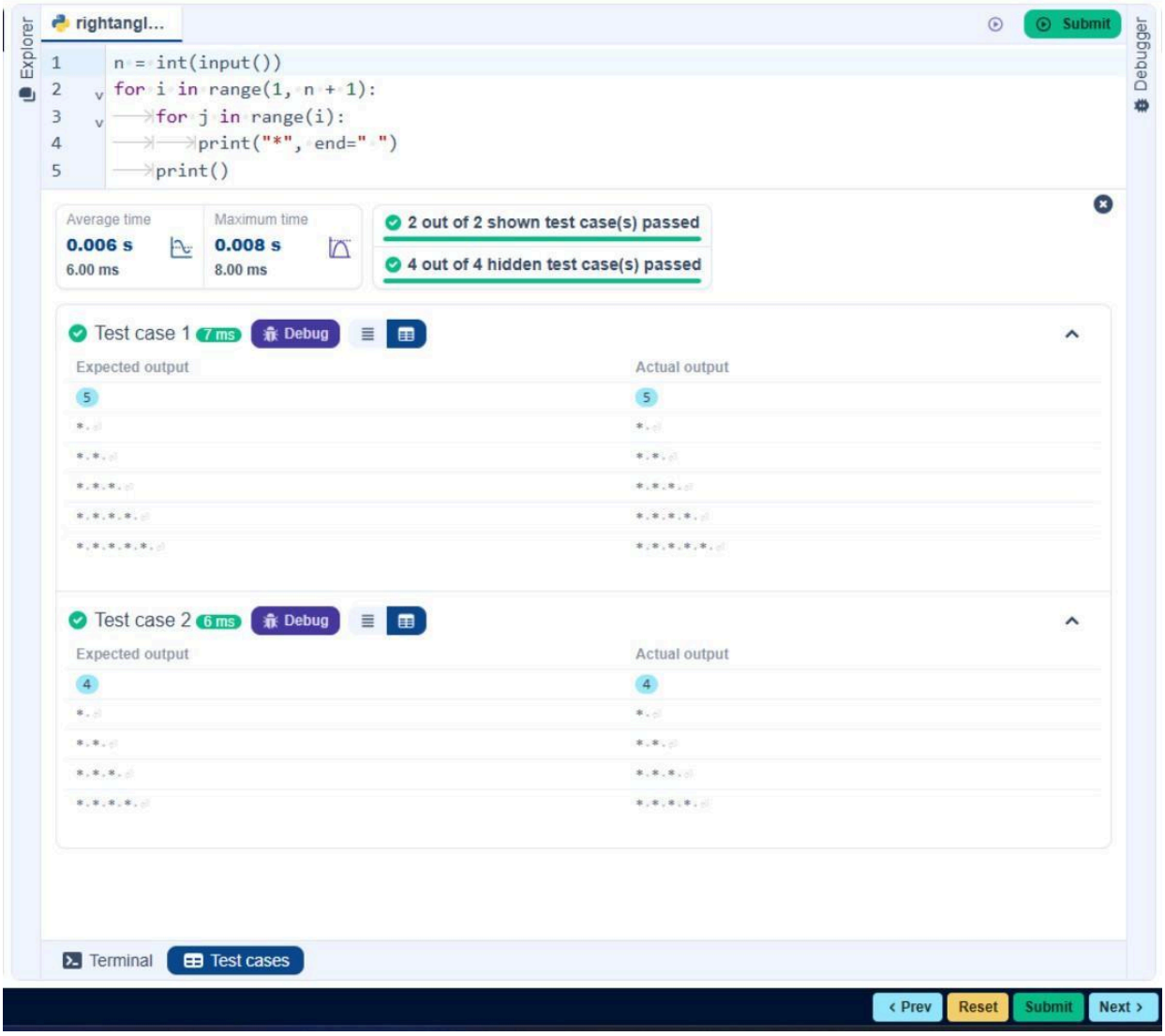
The input is an integer, representing the number of rows in the pattern.

Output Format

The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

```
n = int(input())
for i in range(1, n + 1):
    for j in range(i):
        print("*", end=" ")
    print()
```



1.2.4. Pattern - 2

01:10

Write a Python program to print a right-angled triangle pattern of numbers.

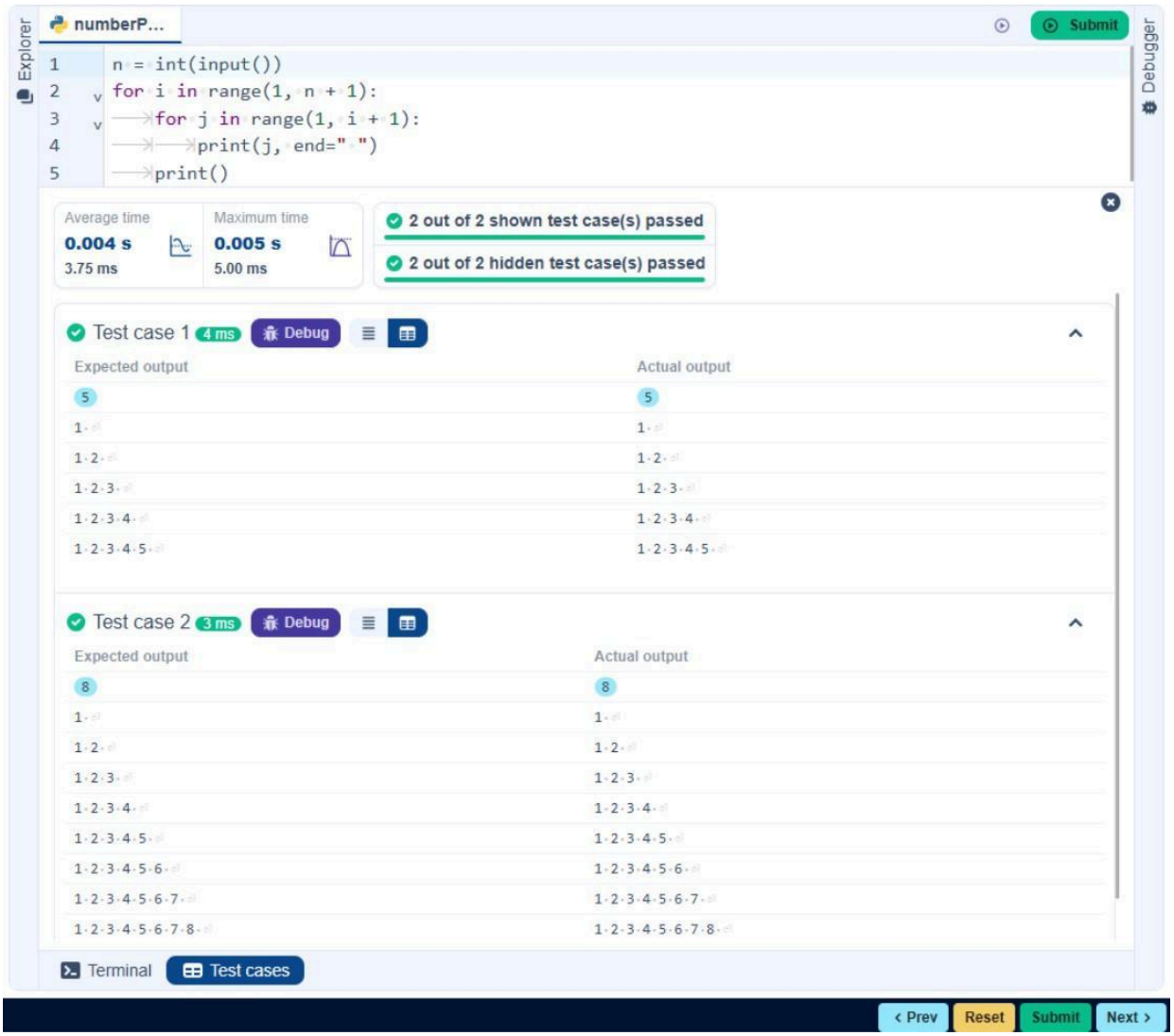
Input Format:

The input is an integer, representing the number of rows in the pattern.

Output Format:

The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

Refer to the displayed test cases for the sample pattern.

```
n = int(input())
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```